

Shaft Design Code

Design of Machine Elements

Department of Mechanical and Aerospace Engineering
University of California, San Diego
MAE 190

William Harris

Introduction:

In this project, I developed a MATLAB code to automate the design of rotating shafts that are subjected to combined bending and torsional loads. One of the main challenges in shaft design is that many parameters depend on each other - you need the diameter to find stress concentration factors, but you need those same factors to find the diameter. This makes it difficult to design shafts by hand, as it takes multiple iterations, requiring constant reference to charts, graphs, and tables while completing numerous calculations. The traditional approach of doing these calculations manually is not only time-consuming but also prone to errors, especially when interpolating values from design charts. Instead of having to do all the calculations by hand, why not have a program handle all of these interdependencies automatically?

My shaft design code addresses these limitations by automating the entire process, from material selection through to final diameter calculation. The program uses computational methods to handle complex calculations, interpolate stress concentration factors, and manage the iteration process, all while ensuring we meet our safety requirements.

Results:

The program is built around several key components that work together to automate the shaft design process. First, I created a flexible material selection system where users can either pick from standard ASTM steels or input their own material properties. The program accepts all the necessary loading conditions (bending moments and torques) and can work in either Imperial or SI units. Rather than asking users to look up stress concentration factors from charts, I implemented interpolation functions that calculate K_t and K_{ts} values based on the D/d and r/d ratios input by the user. The code also calculates notch sensitivity dynamically based on the material's ultimate strength.

The core of the program is its iteration loop, which starts with an initial diameter guess and refines it until we meet our design requirements. In each iteration, the code recalculates all diameter-dependent factors - the size factor k_b gets updated based on the current diameter, stress concentration factors are interpolated using the current D/d and r/d ratios, and notch sensitivity is adjusted. Then it uses the DE-Gerber formula to calculate a new diameter. After each iteration, the code checks two things: has the diameter converged (changed by less than .1%), and are we meeting our target safety factor (within 1%). If either check fails, the code adjusts the nominal safety factor proportionally based on how far we are from our target, and starts another iteration.

For the endurance limit calculation, I included all the Marin factors (k_a through k_e). The size factor updates automatically as the diameter changes during iteration, and the reliability factor is set based on the user's requirements. The whole process uses the DE-Gerber criterion to calculate the diameter while checking both fatigue and yielding safety factors.

When testing the code with the proof-of-concept problem, I used a machined steel shaft with mean bending moment of 3750 lb-in, alternating bending moment of 2250 lb-in,

mean torque of 1250 lb-in, and alternating torque of 1250 lb-in. The material had an ultimate strength of 82 ksi and yield strength of 58 ksi. The geometry required a D/d ratio of 1.2 and r/d ratio of 0.1, with 99.9% reliability and a safety factor of 1.5. The code calculated a minimum shaft diameter of 1.5386 inches, with a fatigue safety factor of 1.5046 and a yielding safety factor of 2.3555. It took just 5 iterations to reach these results.

Discussion:

The code converges pretty reliably thanks to two main checks: one looking at how much the diameter changes between iterations (within 0.0001) and another making sure we hit our target safety factor (within 0.01). Most problems will converge within 5 iterations, which is much faster than doing it by hand. The iteration process is particularly interesting because it needs to balance multiple factors simultaneously - as the diameter changes, several other parameters like the size factor and stress concentration factors also update, which in turn affects the required diameter.

What makes the iteration loop especially tricky is handling the circular dependencies. For example, if the diameter increases, the size factor k_b typically decreases, which reduces the endurance limit and suggests the user needs an even larger diameter. But a larger diameter also changes the stress concentration factors, which might push us in the opposite direction. The code handles these competing effects by using proportional adjustment of the safety factor - if it's 10% below our target safety factor, we increase the nominal safety factor by 10% for the next iteration. This helps prevent the solution from oscillating or diverging.

Another challenge was deciding when to stop iterating. Initially, I just checked if the diameter had converged, but I found that sometimes we'd get a stable diameter that didn't quite meet our safety factor requirements. That's why I added the dual convergence criteria - now requiring both the diameter and safety factor to stabilize before we accept the solution. The code also includes a maximum iteration limit (set to 100) to prevent infinite loops, though in practice it rarely needs more than 5 iterations.

An interesting observation from the proof-of-concept test was the difference between the hand-calculated diameter of 1.43 inches and my code's result of 1.54 inches. This 7% difference comes from how the code strictly enforces the safety factor requirement. In the hand calculations, we usually accept the initial diameter that comes from the DE-Gerber equation, which often gives a safety factor slightly below target. My code, however, continues to iterate and adjust the diameter until it achieves exactly what we're looking for. This highlights one of the key advantages of using computational methods - we can easily iterate until we meet all design requirements exactly, rather than accepting a close-enough solution.

I also added some error checking and input validation to reduce the chance of user error and handle edge cases when interpolating stress concentration factors. The code actively adjusts its calculations to meet the desired safety factor, and it warns the user if it can't meet the safety requirements or if there might be convergence issues. This makes the program much more reliable in practice, as it can catch potential problems before they lead to design errors.

Conclusion:

My shaft design code successfully automates what would otherwise be a tedious manual process. It eliminates the need to constantly reference charts and tables, ensures safety requirements are met, and handles both standard and custom materials. The code is also pretty good at catching user errors and providing helpful feedback. The modular structure makes it easy to modify or expand, which has already proved useful when adding new features during development.

Looking ahead, there are several ways I could improve the code. It would be useful to add other failure criteria like modified Goodman, include shaft deflection analysis, handle multiple diameter changes, and maybe add a GUI. These additions would make the program even more versatile and useful for a wider range of design scenarios.

Overall, I think the code turned out to be a useful tool that makes shaft design much faster and more reliable. The automated approach not only saves time but also reduces the potential for human error in the complex calculations involved. The fact that it strictly enforces safety requirements while optimizing dimensions makes it particularly valuable for applications where precision is essential. Its structure makes it easy to add new features as needed, which could be helpful for future projects or modifications.